

Segment Tree Mastery: Comprehensive Revision Guide

1. Foundational Theory and Concepts

Definition and Purpose

A **Segment Tree** is a sophisticated data structure designed to perform range queries and updates with logarithmic efficiency. It is the architect's choice for scenarios involving dynamic arrays where interval data (sums, minimums, maximums) must be recalculated frequently.

Why Segment Trees?

- **The Prefix Sum Trade-off:** While a Prefix Sum array offers $O(1)$ time for range queries, it suffers from $O(N)$ update costs. The Segment Tree provides a balanced $O(\log N)$ for both, making it essential for mutable data.
- **Logarithmic Scaling:** By dividing the array into a binary tree structure, we reduce $O(N)$ brute-force scans to $O(\log N)$ traversal.
- **Versatility:** Beyond simple sums, they can store indices (for RMQ), GCDs, or even custom objects to solve complex geometric or competitive programming problems.

Mathematical Basis: The $4 \times N$ Array Size

To implement a Segment Tree as an array, we allocate a size of $4 \times N$, where N is the number of elements in the input.

- **Instructor Note:** A perfectly balanced binary tree requires only $2N-1$ nodes. However, since we use a 0-indexed array where a node at index i has children at $2i + 1$ and $2i + 2$, the tree can have "gaps" in its array representation if N is not a power of 2. The $4 \times N$ rule is a safety buffer to prevent "Index Out of Bounds" errors.

Recursive Structure

- **Root:** Represents the total range $[0, N-1]$.
- **Internal Nodes:** If a node covers $[L, R]$, the left child covers $[L, \text{mid}]$ and the right child covers $[\text{mid}+1, R]$, where $\text{mid} = (L+R)//2$.
- **Leaf Nodes:** Represent individual elements where $L == R$.

2. Standard Implementation Patterns (Python)

Watch Out: When implementing child logic, always use $2*i + 1$ (Left) and $2*i + 2$ (Right). A common point of failure is mixing 1-based indexing logic ($2i, 2i+1$) into a 0-based array.

Python

```
class SegmentTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (4 * self.n)
        self.build(data, 0, 0, self.n - 1)
```

Python

```
    def build(self, data, i, l, r):
        if l == r:
            # Base Case: Leaf node stores the value or index
            self.tree[i] = data[l]
            return

        mid = l + (r - l) // 2
        self.build(data, 2 * i + 1, l, mid)
        self.build(data, 2 * i + 2, mid + 1, r)

        # Instructor Variation: Change '+' to min() or max() for
RMQ
        self.tree[i] = self.tree[2 * i + 1] + self.tree[2 * i + 2]
```

Python

```
    def query(self, q_start, q_end, i, l, r):
        # Case 1: Out of range (Return neutral value: 0 for sum,
inf for min)
        if r < q_start or l > q_end:
            return 0

        # Case 2: Complete overlap
```

```

if l >= q_start and r <= q_end:
    return self.tree[i]

# Case 3: Partial overlap
mid = l + (r - l) // 2
return (self.query(q_start, q_end, 2 * i + 1, l, mid) +
        self.query(q_start, q_end, 2 * i + 2, mid + 1, r))

```

Python

```

def update(self, idx, val, i, l, r):
    if l == r:
        self.tree[i] = val
        return

    mid = l + (r - l) // 2
    if idx <= mid:
        self.update(idx, val, 2 * i + 1, l, mid)
    else:
        self.update(idx, val, 2 * i + 2, mid + 1, r)
    self.tree[i] = self.tree[2 * i + 1] + self.tree[2 * i + 2]

```

3. Advanced Technique: Lazy Propagation

Lazy Propagation is the solution for **Range Updates** (e.g., "Add X to all elements in range [L, R]"). Without it, a range update is $O(N \log N)$; with it, we achieve $O(\log N)$.

The "Propagate/Push" Logic

1. Maintain a **lazy** array of size $4 * N$.
2. **Propagate** before any operation: If **lazy[i]** is non-zero, apply it to **tree[i]**, pass the value to **lazy[2i+1]** and **lazy[2i+2]**, then clear **lazy[i]**.

Python

```
def propagate(self, i, l, r):
    if self.lazy[i] != 0:
        # Update current node (e.g., for sum: self.lazy[i] * (r - l + 1))
        self.tree[i] += self.lazy[i]
        if l != r: # Not a leaf: push to children
            self.lazy[2 * i + 1] += self.lazy[i]
            self.lazy[2 * i + 2] += self.lazy[i]
        self.lazy[i] = 0
```

Python

```
def update_range(self, q_l, q_r, val, i, l, r):
    self.propagate(i, l, r)
    if r < q_l or l > q_r: return

    if l >= q_l and r <= q_r:
        self.lazy[i] += val
        self.propagate(i, l, r)
        return

    mid = l + (r - l) // 2
    self.update_range(q_l, q_r, val, 2 * i + 1, l, mid)
    self.update_range(q_l, q_r, val, 2 * i + 2, mid + 1, r)
    self.tree[i] = max(self.tree[2*i+1], self.tree[2*i+2]) #
```

Example for Max-Tree

4. Strategic Pattern: Finding the "Leftmost" Element

Used in problems like LC 2940 and 3479, this pattern searches for the first index meeting a condition in $O(\log N)$.

The Design Pattern

1. **Capability Check:** If `tree_max[i] < threshold`, the range [L, R] cannot contain the value. Return -1 immediately.
2. **Base Case:** If `L == R` and the condition is met, return L.
3. **Left-First Recursion:** Recurse into the **left child** first. If it returns a valid index, return it.
4. **Right-Child Contingency:** Only recurse into the **right child** if the left child returns -1.

5. LeetCode Problem Deep Dive

- **LC 307: Range Sum Query - Mutable:** Implementation of the basic `build/query/update` pattern.
- **LC 2179: Count Good Triplets in an Array:**
 - **Strategy:** Use the Segment Tree as a frequency tracker.
 - **Key Math:** For an index `i` in `Nums1` and its index `idx` in `Nums2`:
 1. **Left Common Count:** Query Segment Tree for range `[0, idx]`.
 2. **Left Uncommon Count:** `i - Left Common Count`.
 3. **Right Common Count:** `(\text{Elements after } idx \text{ in } Nums2) - \text{Left Uncommon Count}`. Formula: $(N - 1 - idx) - (i - \text{Left Common Count})$.
- **LC 2940: Find Building Where Alice and Bob Can Meet:**
 - **Strategy:** RMQ + Binary Search. Use the Segment Tree to find the **leftmost index** `j > \max(A, B)` where `Heights[j] > \max(Height[A], Height[B])`.
- **LC 3477/3479: Fruits Into Baskets II & III:**
 - **Strategy:** Segment Tree on the Baskets array. Use the "Leftmost Search" pattern to find the first basket with capacity `>= fruit`. Use a point update to set the basket capacity to -1 once used.
- **LC 3721: Longest Balanced Subarray II:**
 - **Strategy:** Transform to prefix sums (+1 for even, -1 for odd).
 - **Lazy Prop Role:** When a duplicate element is encountered, use Lazy Propagation to handle a range update that "corrects" the prefix sums by removing the element's previous contribution across the range.

6. Critical Revision Notes & Tips

1. **The Mapping Requirement:** For problems like LC 2179 and 2940, you **must** build an `unordered_map` (Value $\rightarrow$ Index in `Nums2`) before starting. The Segment Tree usually operates on these mapped indices.
2. **Neutral Values:** Always initialize your `query` returns with the operation's identity:
 - Sum: `0`
 - Min: `float('inf')`
 - Max: `float('-inf')`

3. **0-indexed vs 1-indexed:** LeetCode is 0-indexed. If a problem uses 1-based indexing, convert it immediately to avoid $2i$ vs $2i+1$ confusion.
4. **Modular Coding:** Keep **propagate**, **update**, and **query** distinct. Mixing propagation logic into the update function without a helper is the fastest way to introduce bugs in competitive environments.

7. Complexity Reference Table

Operation	Time Complexity	Space Complexity	Expert Notes
Tree Construction	$O(N)$	$O(N)$	Requires 4 times N array for safety.
Point Update	$O(\log N)$	$O(1)$	Direct descent to leaf.
Range Update (Standard)	$O(N)$	$O(1)$	Avoid this; use Lazy Propagation.
Range Update (Lazy)	$O(\log N)$	$O(N)$	Total $O(Q * \log N)$ for Q queries.
Range Query	$O(\log N)$	$O(1)$	Sum, Min, Max, or GCD.